Redis clone (C++ server + Python client/tooling)

Area of focus	Concepts covered	Section requirements	Estimated hours	Specific chapters to reference
• Tooling • “modern C++ baseline” • Python baseline	• build systems • testing • RAII • move semantics • error handling • debugging • packaging	☑ ~~CMake project layout~~	3	• **A Tour of C++**: *The Basics*, *Error Handling*, *Pointers and Containers* • **Effective Modern C++**: *CHAPTER 4 The Smart Pointer Idiom*; *CHAPTER 7 The Concurrency API* (for later) • **Python Crash Course (3e)**: *Getting Started*; *Testing Your Code*
		☑ ~~gtest + test runner~~	3	
		☑ ~~clang-format/clang-tidy~~	2	
		☑ ~~sanitizers (ASan/UBSan)~~	2	
		☑ ~~basic logging~~	2	
		☑ ~~Python venv + pytest~~	2	
		☑ ~~“hello server” + “hello client” smoke test~~	8	
• TCP server skeleton • RESP2 framing	• networking sockets • byte streams • parsing • framing/protocol design10:00am	☑ ~~Blocking TCP server that accepts multiple clients (one at a time initially)~~	10	• **Beej**: *What is a socket?*; *System Calls or Bust* **CSAPP**: *System-Level I/O*; *Network Programming* • **Redis protocol spec**: RESP framing rules/examples (Redis)
		☐ incremental RESP2 parser/serializer (handles partial	10	

		reads)		
		☐ command tokenization	4	
		☐ round-trip tests (golden vectors)	4	
• Core KV engine (strings) • keyspace • TTL	• data structures • time • API design • correctness	☐ Implement: PING, ECHO, GET, SET (NX/XX optional), DEL, EXISTS, INCR/DECR	3	• **A Tour of C++**: *Containers*, *Algorithms*, *Utilities* • **DDIA**: *Reliable, Scalable, and Maintainable Applications*; *Data Models and Query Languages*
		☐ TTL via EXPIRE/PTTL/PERSIST	8	
		☐ monotonic clock usage	7	
		☐ deterministic unit tests	7	
• Single-threaded event loop (nonblocking I/O)	• nonblocking I/O • readiness notification • buffering • backpressure	☐ Switch to nonblocking sockets	5	• **Beej**: *Slightly Advanced Techniques* (select/poll patterns) **CSAPP**: *System-Level I/O*; *Network Programming*
		☐ poll/select (or epoll/kqueue if Linux/macOS)	10	
		☐ per-connection	6	

		read/write buffers		
		☐ support pipelining	5	
		☐ graceful disconnects	2	
		☐ “slow client” tests	2	
Data types v1 ● (lists/hashes/sets/zsets subset)	● internal encodings ● algorithmic complexity ● API surface	☐ LIST: LPUSH, LPOP, LRANGE	6	● **A Tour of C++**: *Containers*, *Algorithms*, *Ranges* ● **DDIA**: *Storage and Retrieval*
		☐ HASH: HSET, HGET, HGETALL	6	
		☐ SET: SADD, SREM, SISMEMBER	6	
		☐ ZSET: ZADD, ZRANGE by score (optional)	8	
		☐ document complexity	3	
		☐ add fuzz-ish tests for edge cases	6	
Persistence v1: ● AOF	● durability tradeoffs	☐ Append-only file	4	● **Redis persistence docs** (AOF semantics +

(append-only log)	• write-ahead logging • fsync policies	for writes		tradeoffs) (Redis) • **CSAPP**: *System-Level I/O* • **OSTEP**: file interface + crash consistency chapters (TOC)
		☐ configurable fsync policy (always/everysec/never)	4	
		☐ replay AOF on startup	5	
		☐ corruption detection (checksum or strict parse)	4	
		☐ Crash recovery test (kill + restart)	3	
• Python client • integration harness • baseline benchmark report	• client/server • DX • reproducibility • performance measurement	☐ Minimal redis-cli-like Python tool + library	5	• **Python Crash Course (3e)**: *Files and Exceptions*; *Testing Your Code* • **CSAPP**: *Optimizing Program Performance* (methodology)
		☐ integration suite that compares behavior vs real Redis for supported commands	7	
		☐ benchmark script (throughput + p50/p99 latency)	5	

		☐ README with how to run	3	
Persistence v2: • RDB snapshot • AOF rewrite	• snapshotting • copy-on-write strategies • background work	☐ RDB snapshot format (your own simplified spec is fine if documented)	8	• **Redis persistence docs** (RDB vs AOF) (Redis) • **OSTEP**: *Crash Consistency / Log-structured File Systems* (TOC)
		☐ background snapshotting (threaded is OK)	8	
		☐ AOF compaction/rewrite	8	
		☐ startup chooses best recovery path	4	
		☐ crash-safe temp-file + atomic rename	4	
		☐ Corrupt AOF tail test	3	
• Concurrency for background jobs (without breaking the event loop)	• threads • task queues • synchronization • atomics	☐ Job system for snapshot/rewrite	8	• **C++ Concurrency in Action**: *Managing threads*; *Sharing data between threads*; *Synchronising concurrent operations* • **Effective Modern C++**:
		☐ bounded queue	4	
		☐ clean shutdown	5	

		☐ prove thread safety boundaries (what's single-threaded vs shared)	5	*CHAPTER 7 The Concurrency API*
		☐ avoid data races (TSan run in CI)	3	
Replication v1 • (leader → follower)	• distributed systems basics • streaming logs • offsets	☐ Replica connects and performs handshake	6	• **Redis replication docs** (conceptual model) (Redis) • **PSYNC command reference** (handshake/stream initiation) (Redis) • **DDIA**: *Replication*; *The Trouble with Distributed Systems*
		☐ Full sync: leader sends snapshot, replica loads it	12	
		☐ Command stream: leader sends writes to replica	10	
		☐ Track replication offset on both sides	5	
		☐ Basic reconnection behavior	5	

		☐ Replication integration tests (kill/restart replica)	7	
● Pub/Sub ● blocking operations	● coordination ● condition variables ● fairness ● wakeups	☐ Implement SUBSCRIBE/UNSUBSCRIBE/PUBLISH	6	● **OSTEP**: *Condition Variables* / *Semaphores* / *Event-based Concurrency* (TOC) ● **C++ Concurrency in Action**: sync primitives chapters (ToC)
		☐ at least one blocking op (e.g., BLPOP) with timeouts	8	
		☐ ensure event loop doesn't stall	5	
		☐ soak tests with many clients	6	
Transactions v1 ● (MULTI/EXEC) ● basic isolation story	● atomicity ● queuing ● failure modes	☐ MULTI queues commands per-connection	5	● **DDIA**: *Transactions*
		☐ EXEC runs atomically against single-threaded loop	5	
		☐ DISCARD	2	
		☐ define error semantics	3	

		☐ tests that mirror Redis behavior for your subset	5	
• Memory • eviction • TTL performance	• caching policy • measurement • memory accounting	☐ Memory accounting per key	5	• **Redis eviction docs** (policy guidance) • **CSAPP**: *The Memory Hierarchy* / *Optimizing Program Performance*
		☐ maxmemory	3	
		☐ implement one eviction policy (LRU or LFU approximation)	8	
		☐ prove eviction correctness via tests	5	
		☐ report perf + memory overhead	4	
• Observability • operability	• production readiness • introspection • debugging	INFO-style stats endpoint	4	• **DDIA**: operability/maintainability themes
		Per-command latency sampling	4	
		Structured logging (spdlog, properly used)	3	
		Config file + runtime config subset	4	

		Health check endpoint	2	
		Clear runbook in README	3	
• Performance engineering “FAANG signal” package	• profiling • benchmarking • bottlenecks • systems thinking	☐ CPU profiling report (perf or gprof)	6	• **CSAPP**: *Optimizing Program Performance*
		☐ Memory profiling report	4	
		☐ Benchmark methodology documented	3	
		☐ Before/after optimizations with graphs	10	
		☐ Document tradeoffs (single-threaded loop vs worker pool)	4	
		☐ Fix top 2-3 bottlenecks	8	
Hardening: • fuzzing • fault injection • CI gates	• reliability • testing strategy • defensive engineering	☐ Fuzz RESP parser (libFuzzer/AFL+ + optional)	5	• **OSTEP**: bug classes + failure thinking (TOC) • **CSAPP**: *Exceptional Control Flow* (signals/errors mindset)

		☐ chaos tests (drop connections, corrupt AOF tail, crash during rewrite)	5	
		☐ CI gates: unit + integration, sanitizers on every commit	3	
		☐ TSan nightly run, clean	2	
• Final “internship-ready ” deliverables	• communication • design docs • reproducibility	☐ Architecture doc (event loop, storage engine, persistence, replication)	4	• **DDIA**
		☐ correctness spec for supported commands	3	
		☐ Performance report with graphs (before/after optimizations)	3	
		☐ Docker image + one-command local demo	4	

		☐ "What I'd build next" section showing production Redis awareness	2	
		☐ README with architecture diagram	2	
Total: 443 hours, 411 hours left				